

What Changed in 2026 — SQL Interview Intel Report

Refresh window: H2 2025 → H1 2026. Last compiled: May 2026.

This file is the field intel layer of the pack. It is not a tutorial. It is a short, dense readout of what we have been hearing from candidates and hiring managers over the last twelve months — what is being asked more, what is being asked less, what is showing up that did not show up a year ago, and what you should triage if you are interviewing this week.

If you came into this pack worried that SQL prep material gets stale fast, this section is the answer to that worry. We refresh it every six months. Everything below is keyed to rounds happening between June 2025 and May 2026 in the Indian market.

Executive Summary — the five shifts that matter most

If you only read one page of this file, read this one. These are the five shifts that have most materially changed the shape of a 2026 SQL interview versus a 2024 one.

- **Window functions are now a baseline expectation, not a senior topic.** ROW_NUMBER, RANK, LAG, LEAD, and a basic SUM OVER (PARTITION BY ...) frame are now appearing in fresher and 0-to-2-year rounds at both Service MNCs and product cos. A year ago, most freshers could get through Round 1 without writing one. That window has closed.
- **Snowflake-specific questions are surging in product co and data role rounds.** Clustering keys, virtual warehouse sizing, time travel, zero-copy cloning, and the materialized view vs dynamic table tradeoff are now showing up in roughly half of mid-level product co rounds where the stack involves a warehouse. Eighteen months ago these were specialist questions; now they are general data engineer screening material.
- **MERGE INTO is replacing the manual INSERT-or-UPDATE pattern in Postgres conversations.** Postgres 15 made MERGE generally available, and interviewers have caught up. If a candidate writes a hand-rolled upsert when MERGE would do, it is being flagged as a knowledge gap in roughly the same way that not knowing JOIN syntax used to be.
- **Isolation level questions are moving earlier in the interview funnel.** Read Committed vs Repeatable Read vs Serializable — with concrete scenario framing rather than rote definitions — is now appearing in Round 1 or Round 2 at fintech and payments cos. Previously this was a Round 3 system design adjacent topic.

- **JSON column patterns have crossed the line from nice-to-know to expected.** JSONB in Postgres, JSON in MySQL 8, and the equivalent in SQL Server 2022 are now interview-table-stakes. The question is no longer "do you know JSONB exists" but "given this JSONB column, write a query that filters on a nested path and returns the matching keys."

A sixth shift worth flagging but slightly behind the others in volume: **deadlock and lock-wait scenario questions are climbing**, particularly at payments and logistics product cos where the on-call surface area is wide. We have moved this into the body of the report rather than the top five because the volume is still smaller than the five above, but the trajectory is steep.

A few framing notes before the body of the report:

- These shifts are biased toward the Indian market between June 2025 and May 2026. US-market data, where we have visibility, mostly agrees on the direction of these shifts but the magnitudes differ. The Snowflake surge, for example, is roughly six months ahead in the US.
- "Service MNC" in our taxonomy means TCS / Infosys / Wipro / Accenture / Cognizant / HCLTech for the purposes of this report. "Product co" means Indian-headquartered consumer or B2B product companies. "GCC" means India-based captive centres of US-headquartered firms.
- We weight signal by recency. A pattern we last heard about in November 2025 is given less weight than one we have heard about in March or April 2026. This means the trends in this report skew toward the most recent six months.
- The single biggest meta-shift behind all of these specific shifts is that interviewers have decided their question banks were stale, and have been actively refreshing them. That refresh started roughly twelve months ago and is still in motion. Expect another smaller wave of shifts in the next six months as the refresh propagates further.

How we source this

We track patterns we hear about — nothing more, nothing less. The signal comes from a few places and we want to be straightforward about what those are so you can calibrate.

The main input is candidate-reported interview content. Candidates who use our prep material tell us what they were actually asked after their rounds. That stream gives us pattern data — which question shapes are recurring across companies, which are dying off, which are new. We do not have access to interviewer panels or hiring rubrics directly. We do not publish question-for-question leaks; we aggregate the *shape* of what is being asked.

The second input is hiring manager and senior engineer conversations — people on our network who run rounds at the companies in this report. They tell us what they have added, dropped, or reweighted in the last quarter. This is qualitative, not quantitative.

The third input is product and engine release notes from the major SQL engines (Postgres, MySQL, SQL Server, Snowflake, BigQuery). When a feature like MERGE INTO graduates to general availability, we can

be fairly confident it will start appearing in interview questions within twelve months. That correlation has held up well.

We refresh this file every six months. Anything older than twelve months in here has been re-validated against current candidate reports for this refresh. We will not pretend to a precision we do not have — when we say "appearing in roughly half of rounds," we mean we have heard this from enough candidates that we are reasonably confident, not that we ran a survey with a p-value.

Service-MNC patterns 2026

Service MNCs — TCS, Infosys, Wipro, Accenture, Cognizant, HCLTech — run high-volume hiring processes with relatively standardized question banks. The good news for candidates is that the question shapes are predictable. The shift over the last twelve months is that the question banks have been refreshed, and the new patterns are visibly more practical and less rote-definition than what we tracked in 2024.

TCS

TCS rounds at the 2-to-5 year experience band are now consistently including the following patterns, where "consistently" means we have heard them in the majority of rounds reported back to us.

- ⚡ **Second-highest salary by department using DENSE_RANK.** This used to be a "self-join trick" question; it has migrated to a window function question and the expected solution is now `DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC)`. Candidates who write the old correlated subquery version are not being marked wrong, but they are being asked the follow-up "can you do it with a window function?" with notable frequency.
- **Delete duplicates from a table where there is no surrogate key.** The expected answer increasingly involves a CTE with `ROW_NUMBER() OVER (PARTITION BY <natural key cols> ORDER BY <something>)` and a DELETE on rn > 1. The older GROUP BY + EXISTS pattern is still accepted but considered the long way around.
- **Convert rows to columns without using PIVOT.** This is becoming a litmus test for whether the candidate genuinely understands CASE expressions inside aggregates. The framing is usually "pivot monthly sales into columns" — and the interviewer is checking whether you reach for `SUM(CASE WHEN month = 'Jan' THEN amount END)` reflexively.
- **Find Nth highest salary with a CTE-based approach.** TCS has historically loved this question; the shift in 2026 is that the accepted answer now uses a CTE and window function rather than the older `LIMIT 1 OFFSET N-1` pattern. Both still work; interviewers are explicitly probing for the CTE-with-window version.

- **Date-range gap detection.** Given a table of subscription periods, find gaps. This is new in 2026 rounds — we did not see it materially in 2024 TCS data — and it is being used to surface candidates who can think in terms of window functions chained together rather than candidates who only know SELECT.

Infosys

Infosys has historically leaned into procedural SQL more than the others. That is still true in 2026 but the balance is shifting.

- **Recursive CTEs for hierarchy traversal.** Employee-manager hierarchy queries are now expected to be written with a recursive CTE. Candidates who only know how to do it with self-joins for two levels are being correctly identified as candidates who have not written this before.
- **CASE in ORDER BY.** Sort customers so that VIPs appear first, then by last_purchase_date. This is showing up in roughly a third of mid-level rounds we hear about.
- **Window functions with PARTITION BY for running totals.** Running totals as a windowed SUM rather than a self-join. The interviewer typically prompts "without using a self-join" if the candidate reaches for the old pattern.
- **GROUP BY with HAVING for cohort filtering.** "Find customers who placed orders in every month of Q1 2026." The expected pattern uses `HAVING COUNT(DISTINCT month) = 3` and is now considered a baseline expectation rather than a clever trick.
- **EXISTS vs IN, with a concrete reasoning question.** Infosys has been explicitly asking candidates to explain when EXISTS is preferable to IN, with a NULL-handling angle. This is new flavour for 2026 rounds.

Wipro

Wipro's mid-level data engineer rounds have absorbed more analytics-engineering content over the last year.

- **NTILE for percentile bucketing.** Bucket customers into deciles by spend. We did not see this much before; it is now appearing.
- **LAG/LEAD for month-over-month comparison.** Calculate MoM growth for a metric. The expected solution uses LAG and a divide; candidates who try to do it with a self-join on month-1 are being asked to try again with a window function.
- **CROSS APPLY (where the stack is SQL Server) or LATERAL (where Postgres).** Wipro rounds have started probing whether candidates have ever written a correlated row-by-row computation using LATERAL. It is not yet a majority-of-rounds question, but it is climbing.
- **Index design for a given query plan.** Show me the indexes you would create to support this query. This is a shift away from rote "what is a clustered index" definitions and toward applied reasoning.

Accenture

Accenture's pipeline has tilted noticeably toward cloud-warehouse-adjacent SQL.

- **Snowflake-flavour SQL.** Even when the role is not explicitly Snowflake, Accenture interviewers are increasingly asking questions like "how would this query behave in Snowflake vs Postgres" or "what is a clustering key." If your CV mentions any data platform, expect a touch of this.
- **Window functions with multiple PARTITION BY columns.** "Rank products within category within region." The compound PARTITION BY is now an expected baseline.
- **CTE chains of 3+ levels.** Multi-step queries where each CTE feeds the next. The shift here is that interviewers are evaluating *readability* of the CTE chain, not just correctness.
- **Date math without DATEADD/DATEDIFF aliases.** Expect questions where the dialect is not specified and you need to write date arithmetic that translates. This is checking whether you have written cross-dialect SQL or only one engine's flavour.

Cognizant

Cognizant has historically been the most "interview question bank" of the Service MNCs. The 2026 bank is broader and trickier.

- ⚡ **ROW_NUMBER vs RANK vs DENSE_RANK distinction.** Appearing in the vast majority of mid-level rounds we have heard about. The interviewer typically asks for output for each on the same input data with ties. Candidates who can recite the definitions but stumble on the example output are being filtered out.
- **Self-join for finding pairs.** Find all pairs of employees in the same department earning the same salary. Classic, still being asked, still mostly being botched in the join condition (forgetting the `<` to deduplicate pairs).
- **GROUP BY with multiple aggregates and HAVING.** Composite filter logic post-aggregation. This is being used to test whether candidates conflate WHERE and HAVING.
- **JSON column query (if Postgres or MySQL 8 is on the resume).** Given a JSON column, extract a nested field and filter. New for 2026 at Cognizant; we did not see it in 2024.
- **Transactional scenario question.** Two transactions, what does each see — typically framed around bank balance updates. The expected answer requires the candidate to know what isolation level the engine defaults to.

HCLTech

HCLTech rounds are increasingly leaning into Oracle and SQL Server flavours because of the BFSI client base.

- **Oracle-specific MERGE syntax.** If the role is BFSI-adjacent, expect MERGE INTO questions in Oracle syntax even if your background is Postgres.
 - **Analytical functions with KEEP DENSE_RANK FIRST/LAST.** Oracle-specific construct. Showing up in senior rounds.
 - **Window frame clauses — ROWS BETWEEN vs RANGE BETWEEN.** Climbing in volume. The interviewer is checking whether the candidate has actually written running totals over time-series data and run into the RANGE vs ROWS gotcha.
 - **PIVOT / UNPIVOT in SQL Server.** Still being asked, still expected as built-in syntax answer rather than CASE expression hack at the senior end.
-

Product-co patterns 2026

Product companies — Razorpay, Swiggy, Cred, Meesho, Zerodha, Flipkart, PhonePe — are where the SQL interview has shifted the most in the last twelve months. The depth is up across the board. These rounds are no longer "can you write a JOIN"; they are "can you reason about a query plan, isolation behaviour, partitioning, and write a non-trivial window function chain."

Razorpay

Razorpay's SQL bar is high, particularly on the payments backend side. The 2026 patterns are concentrated on transactional correctness.

- ⚡ **Idempotent payment write under concurrent retry.** Given two concurrent attempts to mark a payment as captured, what SQL pattern do you use to ensure idempotency. The expected answer involves a unique constraint plus INSERT ... ON CONFLICT DO NOTHING (Postgres) or an equivalent. New baseline for 2026.
- **Isolation level question with read skew scenario.** Two queries, one update in between, what does each query see under Read Committed vs Repeatable Read. Appearing in roughly half of mid-level Razorpay rounds.
- **Deadlock reproduction.** Given two transactions updating two rows in opposite order, will they deadlock. This is being asked verbally rather than coded — the interviewer wants to hear you reason about lock acquisition order.

- **Reconciliation query with multi-source LEFT JOIN.** Given a payments table and a settlement table, find unmatched entries on either side. Standard FULL OUTER JOIN with COALESCE pattern. Now baseline expectation.
- **Window function over event stream.** Given a stream of payment events, compute the state of each payment at each event. Requires LAG/LEAD plus a window over partition by payment_id.

Swiggy

Swiggy's mid-level rounds have absorbed more analytics-engineering content than they had two years ago.

- **Order funnel computation.** Given event-level data (search, add-to-cart, checkout, paid), compute conversion rates by step. Expected as a series of CTEs with COUNT DISTINCT user IDs. Now baseline.
- **Cohort retention query.** Given users and their first order date, compute Day-1, Day-7, Day-30 retention. Requires window functions or self-joins on user with date arithmetic. Climbing in volume.
- ⚡ **Surge pricing computation with rolling window.** Compute average price per area over rolling 15-minute windows. Requires `RANGE BETWEEN INTERVAL '15 minutes' PRECEDING AND CURRENT ROW`. We see this in 2026; we did not see it in 2024. Window frame clause literacy is being directly tested.
- **Restaurant ranking within cuisine.** Standard partitioned ranking; expected to use DENSE_RANK with handling for ties.
- **Index design for a hot read path.** Given a query pattern, propose the index. Composite index ordering matters and is being probed.

Cred

Cred's bar on SQL is among the highest in the Indian product co cluster. Their rounds skew toward optimization and correctness rather than feature breadth.

- **EXPLAIN ANALYZE walk-through.** Given a query plan, identify the bottleneck. We have heard this is being used as a working-style interview — they put a real plan in front of you and want to hear how you read it.
- **Avoiding N+1 in a reporting query.** Given a description of a Python service that issues N+1 queries, rewrite as a single SQL query. The candidate is being evaluated on whether they reach for window functions or subqueries to flatten the access pattern.
- **Materialized view vs view tradeoff.** When would you reach for each. New baseline question for 2026; we did not see it as often in 2024.

- **CTE materialization in Postgres 12+.** Cred specifically has been probing whether candidates know that CTEs in Postgres 12+ are no longer always optimization fences. This is niche and senior, but climbing.
- **Partial index design.** Given a table with a status column where 95% of rows are status = 'done', design an index that benefits the 5% pending queries. Partial index with a WHERE clause is the expected answer.

Meesho

Meesho's hiring volume has been high and the SQL bar has firmed up.

- **Seller cohort analysis.** Given seller registration date and monthly GMV, compute cohort GMV curves. Standard cohort SQL with window functions.
- **Catalog deduplication.** Given a products table with near-duplicates on multiple fields, write a query to surface duplicate candidates. ROW_NUMBER over a PARTITION BY composite key.
- **Order cancellation funnel.** Given event data, what proportion of orders are cancelled at each stage. Expected as a CTE chain with conversion rates.
- **Date-bucket aggregation with gap-filling.** Compute daily metric with zero-filled rows on days where no events occurred. Requires a date-spine table or `generate_series` joined LEFT OUTER. Climbing.

Zerodha

Zerodha's SQL bar is shaped by the trading-platform domain. Expect time-series flavour throughout.

- **Tick-data downsampling.** Given second-by-second tick data, compute minute-by-minute OHLC (open, high, low, close). Requires window functions with FIRST_VALUE and LAST_VALUE plus MIN/MAX. We see this as a now-baseline question.
- **Position calculation from trade history.** Given a sequence of buy/sell trades, compute the running position by symbol. Standard SUM OVER (PARTITION BY symbol ORDER BY ts).
- **Brokerage calculation with tiered rules.** Given a tier table and trade amounts, compute the brokerage. Tests whether candidates reach for a join-on-range pattern or a CASE expression — both can be correct depending on the tier table size.
- **Read consistency for a balance-display query.** What guarantees does the user need when they refresh their portfolio view, and what isolation level provides those. Showing up at the senior end.

Flipkart

Flipkart rounds are broad and include classical big-data flavour at the senior end.

- **Top-N per category with ties.** Standard windowed RANK with the ties handled correctly. Climbing.

- **Inventory reconciliation across warehouses.** Multi-table join with aggregation; tests the candidate's ability to keep cardinality straight across joins.
- **Skewed key handling.** Given a join that is slow because of skew on one key (say one seller has 90% of orders), what would you do. Tests salting awareness and warehouse-specific optimization knowledge.
- **Backfill query under partition pruning.** Write a backfill that processes one partition at a time. New flavour for 2026; reflects the shift to partitioned tables being mainstream in their stack.

PhonePe

PhonePe rounds emphasize payments correctness and scale.

- **UPI transaction reconciliation.** FULL OUTER JOIN pattern with COALESCE and NULL handling. Baseline expectation.
- **Fraud-detection rule expressed in SQL.** Given a rule like "more than 5 transactions in 10 minutes from the same device," write the SQL. Window functions with time-based ranges.
- ⚡ **Hot partition mitigation.** Tests warehouse-specific awareness of how to spread writes when one partition is hot. Climbing fast in 2026.
- **Consistent point-in-time read across two tables.** Tests isolation knowledge and may extend to snapshot semantics.
- **Latency budget for a query.** Given a 50ms budget, walk through how you would profile and optimize a query. Tests whether candidates have ever profiled SQL versus only written it.

Data-role company patterns 2026

For analytics engineers and data engineers at companies like InMobi, Dream11, Slice, Jupiter, and Khatabook, the 2026 SQL interview has moved decisively into warehouse and modeling territory. Knowing your engine flavour and your dimensional modeling vocabulary is now table stakes.

Stack-specific SQL

- **Snowflake-flavour SQL is now expected at the majority of these companies.** Even where the production stack is partially Postgres-based, the interview tends to probe Snowflake constructs because that is where the modeling work is happening. Specifically: clustering keys (when and why to declare them), virtual warehouse sizing decisions, query history analysis, time travel for accidental delete recovery, zero-copy clones for dev environments.

- **BigQuery flavour is the alternative.** At companies where BigQuery is the warehouse, expect questions on partitioned + clustered tables (partition by ingestion date, cluster by 4 cols max), slot reservations, scheduled queries, and STRUCT/ARRAY data access patterns.

dbt patterns being probed

dbt is now common enough in the data engineer pipeline that interviewers are asking about it directly.

- **Incremental model design.** Given a fact table that receives append-only events, how would you write the incremental model. Expected to mention `is_incremental()` macro, the merge strategy, and the unique_key argument.
- **Source freshness and tests.** Tests on uniqueness, not_null, accepted_values — and source freshness checks. The interviewer wants to hear that you have written these, not just heard of them.
- **Snapshot strategy.** Type 2 slowly changing dimension via dbt snapshot — when and why. Climbing in volume.
- **Macro usage.** When should you write a custom macro vs inline the SQL. Tests judgement.
- **Model dependency design.** Given a DAG, where would you put the test, where would you materialize vs leave as view. Senior-level question.

Dimension modeling

This was historically a textbook topic; in 2026 it is being asked in concrete-scenario form.

- **Star vs snowflake schema tradeoff for a given fact.** Tests whether the candidate can reason about cardinality and query patterns.
- **SCD Type 2 implementation in SQL.** Write the MERGE that handles new rows, updated rows, and historical preservation. Postgres 15 MERGE is making this question cleaner and more common.
- **Conformed dimensions across multiple facts.** Tests whether the candidate has worked with a real warehouse rather than a textbook one.
- **Fact table grain decisions.** Given a description of business events, decide the grain. The grain decision is the most-asked dimensional modeling question in 2026.

Performance and cost

Cloud warehouses make cost an explicit interview topic.

- **Reduce a query's slot/credit cost.** Given a Snowflake or BigQuery query, propose three changes that would lower its cost. Filter pushdown, clustering, partition pruning, projection narrowing.
- **Materialized view vs dynamic table (Snowflake).** When would you reach for each in 2026. Dynamic tables changed the calculus and interviewers know it.

- **Detecting and fixing data skew in a join.** Senior-level. Tests whether the candidate has handled real-world Snowflake or BQ pain.
-

GCC patterns 2026

GCCs of US firms — JP Morgan India, Walmart Labs, Wells Fargo, Goldman, Target, AmEx, Morgan Stanley — interview to bars set by the US org. The SQL component tends to be heavier on SQL Server, Oracle, and DB2 dialects than the rest of the Indian market. Finance-domain scenarios dominate. The shift over the last year has been toward more complex CTE chains and more pointed point-in-time query questions.

Recurring patterns across GCCs

- **Running balance from a transaction log.** Foundational. Window-function SUM OVER ORDER BY. Now baseline expectation at every GCC we have data on.
- **Point-in-time balance lookup.** Given a transaction log, what was the account balance as of date X. Tests whether the candidate can write the window function with a date filter cleanly.
- ⚡ **As-of join (event table joined to a slowly-changing dimension).** Given an events table with timestamps and a rates table with effective_from/effective_to ranges, join each event to the rate that applied at its timestamp. This is the workhorse query of finance SQL and it is being asked everywhere in 2026. Expected solution involves a join with a BETWEEN range condition, or a LATERAL subquery with an ORDER BY ts DESC LIMIT 1.
- **Multi-currency aggregation.** Convert amounts at the prevailing FX rate as of each transaction date and aggregate. Requires the as-of join plus an outer aggregation.
- **Regulatory report query.** Given a compliance rule (e.g., aggregate exposure by counterparty above a threshold), write the SQL. Tests whether candidates can read a rule and translate to SQL precisely.

JP Morgan India

- **Complex CTE chain with 5+ steps.** A real reporting query. Tests both correctness and readability.
- **Advanced window framing — ROWS BETWEEN, RANGE BETWEEN, named WINDOW clause** (SQL Server 2022 added the named WINDOW clause). Climbing in senior rounds.
- **Pivot of trade data by trade type.** Standard.
- **Recursive CTE for organizational hierarchy.** Expected solution; deep hierarchies are a real-world fixture.

Walmart Labs

- **Inventory reconciliation across stores and DCs.** Multi-table FULL OUTER JOIN with NULL handling on either side.
- **Demand forecasting input query.** Window functions over time-series sales data.
- **Backfill design for a corrupted partition.** Tests warehouse-engineer judgement.

Wells Fargo

- **Stress test scenario aggregation.** Aggregations across portfolio dimensions with stress-test multipliers applied via CASE or join-on-range.
- **CCAR-style regulatory query.** A specific reporting pattern; senior-level. Tests whether the candidate has worked in BFSI.
- **Audit log query for a given record's history.** Window function or recursive CTE.

Goldman

- **Point-in-time pricing query.** Tests as-of join fluency.
- **Position attribution by sector.** Standard aggregation with window functions for share-of-total.
- **Lineage query (which inputs went into this output).** Tests recursive CTE writing.

Target

- **Promo-impact analysis.** Compare metrics across promo and non-promo periods. Window functions plus CASE expressions.
- **Stockout detection across time.** Gap detection on time-series; window functions.
- **Markdown optimization input query.** Aggregation with multiple dimensions and partitioned window functions.

AmEx

- **Cardholder-spend cohort analysis.** Standard cohort SQL.
- **Fraud rule expressed in SQL.** Window functions with time-windowed counts.
- **Authorization-decline analysis.** Reason-code rollup with aggregations.

Morgan Stanley

- **Portfolio-level VaR-input query.** Aggregation across positions with risk-weighting.
- **Trade-reconciliation query.** As-of join with two trade sources.

- **Cross-asset roll-up.** Conformed dimensions in action.

Postgres 15-16 — what's new and being asked

Postgres 15 and 16 shipped features that did not exist eighteen months ago in any meaningful interview-prep sense. They now do. Here are the eight Postgres-specific topics that have been climbing.

1. MERGE became generally available in Postgres 15

MERGE landed as a first-class statement. Before this, the idiomatic upsert pattern was

`INSERT ... ON CONFLICT DO UPDATE`, which is still valid and still common. But MERGE is now the "I know modern Postgres" answer when the requirement involves multiple branching cases (insert-this, update-that, delete-the-third).

Sample question phrasing:

"Given a staging table of customer updates and a target customer table, write a single statement that inserts new customers, updates existing customers if the data has changed, and marks customers as inactive if they are present in the target but absent from staging."

The expected 2026 answer is a MERGE statement with three WHEN clauses.

2. Logical replication improvements in Postgres 16

Postgres 16 added the ability to perform logical replication from standby servers, and various filter improvements. Interview-relevant because data engineers are now expected to know whether their CDC pipeline reads from primary or replica.

Sample question phrasing:

"If your downstream CDC consumer needs to be isolated from the primary write load, what changed in Postgres 16 that lets you do this cleanly?"

Expected answer: logical replication from standbys.

3. JSON path operators @? and @@

Postgres added SQL/JSON path support a few releases back, and adoption has caught up. The `@?` operator tests path existence; `@@` tests a path predicate. These are showing up in interview rounds where the candidate has Postgres on their CV.

Sample question phrasing:

"Given a JSONB column called `metadata` , write a query that returns rows where `metadata.attributes.tags` contains the value 'priority'. Use the SQL/JSON path syntax."

Expected answer:

```
SELECT *  
FROM events  
WHERE metadata @@ '$.attributes.tags[*] == "priority";
```

4. Parallel index builds

Postgres can build B-tree indexes in parallel since Postgres 11, but adoption awareness has been climbing. In 2026 interviews this comes up as a "how would you reduce maintenance window" question.

Sample question phrasing:

"You need to add an index to a 500 GB table during a maintenance window. What knobs would you turn to make this complete faster?"

Expected answer: `max_parallel_maintenance_workers`, plus the trade-off discussion of `CONCURRENTLY` vs not.

5. ANY_VALUE aggregate (Postgres 16)

A small but interview-relevant addition: `ANY_VALUE` returns an arbitrary value from a group. Useful when you have grouped on some columns and want a column that you know is functionally dependent on the group key but don't want to put through `MAX/MIN`.

Sample question phrasing:

"Given a customers table grouped by `customer_id`, how would you fetch the customer name without using `MIN(name)` or `MAX(name)`?"

Expected answer: `ANY_VALUE(name)` .

6. IS DISTINCT FROM semantics

Not new, but newly-asked in 2026. The operator handles NULL correctly where `=` does not. Interview rounds at data-heavy product cos are probing whether candidates have ever been bitten by NULL comparison bugs.

Sample question phrasing:

"You're writing a CDC consumer that needs to detect changes between an old row and a new row. Why is `old.col = new.col` insufficient, and what do you use instead?"

Expected answer: IS DISTINCT FROM handles NULL correctly.

7. Generated columns

Stored generated columns shipped in Postgres 12 but adoption is now mainstream and interviewers have noticed. Showing up at the data engineer level.

Sample question phrasing:

"Given a transactions table with `amount` and `currency`, you want a permanently-stored `amount_usd` column that updates with the source. How do you do this without writing a trigger?"

Expected answer: generated column with STORED keyword.

8. Partitioning improvements

Postgres partitioning continues to mature. Default partitions, list partition pruning, and partition-wise joins are now interview-mainstream rather than fringe.

Sample question phrasing:

"Given a partitioned events table by month, what query writing patterns help the planner prune partitions?"

Expected answer: filter on the partition key directly with literal or stable expressions; avoid functions on the partition key that prevent pruning.

Snowflake & BigQuery surge

Cloud-warehouse-specific SQL has climbed sharply in 2026 interview content. Below are ten topics newly trending, split across Snowflake and BigQuery.

Snowflake

1. Clustering keys. When to declare them, when to leave them auto, what does the clustering depth signal mean.

Sample question phrasing:

"Given a 10 TB table, when would you declare an explicit clustering key, and on which columns?"

Expected discussion: query patterns, cardinality, auto-clustering cost, clustering depth signal as the maintenance trigger.

2. Virtual warehouse sizing and auto-suspend. Why is the warehouse the unit of compute, what is the cost model.

Sample question phrasing:

"Your dashboard query is running on a large warehouse and costing too much. What are the first three things you would investigate?"

Expected discussion: right-size the warehouse, check auto-suspend, separate query and load workloads, evaluate clustering.

3. Time travel. Recovering accidentally dropped data, querying historical state.

Sample question phrasing:

"Someone ran a DELETE without a WHERE clause five minutes ago. How do you recover?"

Expected discussion: AT|BEFORE clause, UNDROP, time travel retention period.

4. Zero-copy cloning. Why is this transformative for dev workflows.

Sample question phrasing:

"How would you give the analytics team a full copy of production for ad-hoc analysis without doubling storage cost?"

Expected discussion: zero-copy clone, the cost-on-divergence model.

5. Materialized views vs dynamic tables. New comparison in 2026. Dynamic tables added declarative pipelines.

Sample question phrasing:

"When would you build a dynamic table instead of a materialized view? What does the refresh model look like?"

Expected discussion: incremental refresh, dependency management, target lag, the move from imperative refresh to declarative.

BigQuery

6. Slot reservations. On-demand vs reserved slots; cost model.

Sample question phrasing:

"Your finance team flagged a \$40K BigQuery bill last month. Walk through how reservations might have prevented this."

Expected discussion: editions, autoscaling reservations, baseline + max slot configuration.

7. Partitioned + clustered tables. The two-step optimization for BQ.

Sample question phrasing:

"Design the partitioning and clustering for an events table that is queried by user_id with a date range filter."

Expected answer: partition by date (ingestion or event), cluster by user_id with at most three additional cluster keys.

8. ARRAY and STRUCT handling. UNNEST queries.

Sample question phrasing:

"Given a row with an ARRAY> field representing line items, write a query that produces one row per line item."

Expected answer: CROSS JOIN UNNEST.

9. Scheduled queries vs scheduled procedures. When to reach for each.

Sample question phrasing:

"You need to run a daily aggregation that has dependencies on three upstream tables. Scheduled query, stored procedure, or external orchestrator?"

Expected discussion: dependency awareness, observability tradeoffs.

10. Secure views and authorized views. Sharing data without exposing the underlying table.

Sample question phrasing:

"How do you let an external project query a filtered subset of your dataset without giving them access to the base table?"

Expected discussion: authorized view, dataset-level grants.

MySQL 8.x shifts

MySQL 8 finally added window functions and CTEs, and the interview market has caught up. If you trained on MySQL 5.7 patterns, your prep is out of date.

- **Window functions.** All standard window functions (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, NTILE, SUM/AVG OVER) are now in MySQL 8. Interviewers are testing whether the candidate has actually written them in MySQL or whether they only know the Postgres syntax. The two are largely identical, but the candidate's fluency is what is being measured.
- **CTEs and recursive CTEs.** Same point. Recursive hierarchy queries are now expected to be doable in MySQL.
- **Invisible indexes.** New shape of question: "you want to drop an index but are not sure if it is used; how would you confirm safely?" Expected answer: mark it invisible first, monitor, then drop. Climbing in mid-level rounds.

- **Descending indexes.** When the query has ORDER BY col DESC, a true descending index avoids the file sort. Tests whether the candidate has profiled real queries.
 - **JSON_TABLE.** Standard SQL/JSON function that converts JSON to relational rows. Showing up at data engineer level.
 - **Histograms.** MySQL 8 added single-column histograms for cardinality estimation on non-indexed columns. Interview-relevant for "the planner is picking the wrong join order, what do you do."
-

SQL Server 2022 shifts

SQL Server 2022 shipped JSON improvements and a small set of T-SQL ergonomics that have made their way into interviews.

- **JSON improvements.** OPENJSON for parsing, JSON_OBJECT and JSON_ARRAY for constructing. Showing up at mid-level. The interviewer expects you to know that SQL Server has caught up with Postgres and MySQL on first-class JSON support.
 - **Ledger tables.** Tamper-evident tables for compliance scenarios. Niche but climbing in BFSI GCCs.
 - **GREATEST and LEAST.** Finally arrived. Interview-relevant because the workaround patterns (CASE expressions or VALUES table) are now the wrong answer when SQL Server 2022 is the stack.
 - **IS [NOT] DISTINCT FROM.** Added in SQL Server 2022. Same NULL-safe semantics as Postgres. Asked in change-data scenarios.
 - **DATE_BUCKET.** Time-series bucketing built in. Replaces a CASE-or-floor pattern.
-

Falling out of fashion

Some patterns are being asked less, or with less weight. If your prep time is limited, deprioritize these.

Pattern	Status	Notes
Basic SELECT and WHERE in mid+ rounds	Largely gone	Still in screening rounds; not in interview rounds for 2+ years experience
Rote normalization theory at senior+	Significantly down	Only asked if framed as a concrete schema design scenario
ACID definition without scenario	Down	"What does ACID stand for" is no longer enough; you will be asked to apply it
Basic INNER vs LEFT JOIN definition	Down at mid+	Asked at junior; expected as muscle memory at mid
Stored procedures as a primary interview topic	Down outside BFSI	Procedural SQL is fading except in GCC and Oracle-heavy shops
Triggers as a "have you used them" question	Down	Now framed as "would you reach for a trigger or a generated column / CDC stream"
Pure DDL syntax recall (CREATE INDEX flags)	Down	Replaced by reasoning questions about which index to create
Subquery vs CTE preference as a stand-alone question	Down	Now embedded inside a larger query design question
GROUP_CONCAT or STRING_AGG as a clever trick	Down	Now expected as a baseline answer, not flagged as advanced
Cursors as a thing to "know"	Down	Asked only to confirm you would NOT reach for one in 99% of cases

The pattern across the table: the interview market has shifted from "do you know this exists" questions toward "can you reason about which option to use and why" questions. The death of trivia questions has been one of the most encouraging shifts of the last two years.

Q1 vs Q3 2026 drift — the last 90 days

Concretely, here is what has shifted in the most recent quarter — between roughly February and May 2026. If you interviewed at the start of the year and again now, here is what you would notice.

- **Dynamic tables (Snowflake) questions appearing where they had not been.** Dynamic tables were not a frequent ask in Q1; by Q2 they are showing up in roughly a third of Snowflake-stack

rounds. The dynamic-tables-vs-materialized-views comparison is now the second most-common Snowflake question after clustering keys.

- **MERGE INTO replacing INSERT ... ON CONFLICT in Postgres rounds.** Interviewers are explicitly probing for MERGE awareness now. Candidates who reach only for ON CONFLICT are being asked the follow-up.
- **Idempotency questions migrating from senior-only to mid-level.** "Make this write idempotent under retry" used to be a senior-systems question. It is now a mid-level question at fintech and payments cos.
- **Cohort retention SQL appearing at non-data roles.** Cohort queries used to be data-engineer interview content. They are now appearing in backend mid-level rounds at consumer product cos, presumably because the business analytics work has spilled into application teams.
- **JSON path operators more prominent.** @@ and @? in Postgres specifically. Q1 saw mostly `-->` and `->` chains; Q2 has seen real SQL/JSON path expressions.
- **Cost-of-query reasoning showing up at GCCs.** Slot cost (BQ) and credit cost (Snowflake) are now part of cloud-warehouse rounds at GCCs that previously kept the interview engine-agnostic.
- **As-of join becoming the workhorse finance SQL question.** It was always common; it is now near-universal at finance GCCs.
- **PIVOT-in-Postgres-by-the-CASE-trick has been re-promoted.** With more questions on monthly KPIs and reporting, the CASE-pivot pattern is back in higher rotation.
- **Read uncommitted as a "trick" question has nearly disappeared.** Interviewers seem to have decided that asking about Read Uncommitted is more confusing than illuminating; almost no one is asking it standalone now.
- **Window frame clause specificity is up.** "ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW" — interviewers want to see candidates write this out rather than rely on the default frame.

What to skip if your interview is next week

If your interview is in seven days, you cannot read everything. Here is the triage.

If you're a fresher to 2 years: - ABSOLUTELY do: joins (all four), GROUP BY + HAVING, basic window functions (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD), the isolation levels at one-line-each (Read Committed / Repeatable Read / Serializable behaviour in one sentence each), the duplicate-deletion pattern, the second-highest-salary pattern (both old and new ways), and what an index is at a one-paragraph level. - SKIP: partitioning theory in any depth, dbt macros, Snowflake-specific topics if

you have no warehouse experience, recursive CTEs beyond the textbook example, query plan reading, MERGE INTO if your stack isn't Postgres 15+.

If you're 2-5 years and applying to product cos: - ABSOLUTELY do: window functions chained together (running totals, MoM, cohort), isolation levels with scenario framing (read skew, lost update, phantom read), the idempotent-write pattern, EXPLAIN ANALYZE basics for whichever engine is your stack, JSON column queries. - SKIP if pressed for time: BigQuery if your stack is Postgres/MySQL only, ledger tables (very niche), DB2-specific features, deep dimensional modeling (only if you're not interviewing for a data role).

If you're applying to a data-role company: - ABSOLUTELY do: dimensional modeling fundamentals (star vs snowflake, grain, SCD Type 2), dbt incremental models and tests, Snowflake or BigQuery patterns depending on which is on the JD, cost reasoning, materialized vs dynamic tables. - SKIP: pure OLTP transaction patterns (you will get one isolation question and that's it), procedural SQL.

If you're applying to a GCC: - ABSOLUTELY do: as-of join, running balance / point-in-time, recursive CTEs for hierarchy, the engine flavour on the JD (Oracle, SQL Server, or DB2), complex CTE chain readability. - SKIP: Snowflake-specific topics unless explicitly mentioned, dbt unless explicitly mentioned, cloud-warehouse cost reasoning.

If you're applying to a Service MNC: - ABSOLUTELY do: ROW_NUMBER vs RANK vs DENSE_RANK distinction, recursive CTE for hierarchy, duplicate deletion, second highest salary, pivot via CASE expression, JSON column query, one or two questions about your specific stack on the resume. - SKIP: Snowflake-deep, dynamic tables, BigQuery, partitioning at depth, dbt at depth.

Cross-cutting advice for next-week mode: - Memorize the three isolation levels and the anomalies each prevents. This single piece of information is being asked across every category now and it is highly compressible. - Be able to write a window function fluently without thinking. The single largest gap between candidates who pass and candidates who don't in 2026 is window-function fluency. - Know what your own resume says. If you put "MERGE INTO" or "dbt" or "Snowflake" on your CV, you will be asked about it. If you don't, you mostly won't be asked the deep version. - Practice writing SQL on a whiteboard or empty editor without autocomplete. Many of these rounds are still done without an IDE. Your hands need to know the syntax.

Good luck. Read the rest of this pack with this intel layered on top — it tells you which questions in the bank to weight heavier this cycle.